

How to Make an End to End Automatic Speech Recognition System with Wav2vec 2.0



Georgian

Follow



Mar 24 · 9 min read

By [Zilun Peng](#), [Akshay Budhkar](#), [Jumana Nassour](#), [Ilana Tuil](#) and [Jason Levy](#).

Thanks to advances in speech recognition, companies can now build a whole range of products with accurate transcription capabilities at their heart. [Conversation intelligence](#) platforms, personal assistants and video and audio editing tools, for example, all rely on speech to text transcription. However, you often need to train these systems for every domain you want to transcribe, using supervised data. In practice, you need a large body of transcribed audio that's similar to what you are transcribing just to get started in a new domain.

Recently, Facebook released wav2vec 2.0 which goes some way towards addressing this challenge. wav2vec 2.0 allows you to pre-train transcription systems using *audio only* — with no corresponding transcription — and then use just a tiny transcribed dataset for training.

In this blog we share how we worked with wav2vec 2.0, with great results.

What is an end-to-end automatic speech recognition system?

Before we dive into wav2vec 2.0, let's take a few steps back to cover a couple of key terms you'll need to understand to see what makes wav2vec 2.0 so special. First, let's look at end-to-end automatic speech recognition systems.

An end-to-end automatic speech recognition (ASR) system takes speech audio waveform and outputs the corresponding text. Traditionally, these systems use Hidden

Markov Models (HMMs), where the speech audio is modeled using a stochastic process. In recent years, deep learning ASR systems have become popular thanks to increased computing power and amounts of training data.

You can measure an ASR system's performance with a word error rate (WER) metric. WER reflects the number of corrections needed to convert the ASR output into the ground truth. Generally, a lower WER means a better quality ASR system.

Ground truth: How are you today John

ASR output: How you a today Jones

This figure is adapted from <https://docs.microsoft.com/en-us/azure/cognitive-services/speech-service/how-to-custom-speech-evaluate-data>

The example above shows how to calculate the WER. We can see that the ASR has made a few errors. It has inserted an “a”, identified “John” as “Jones” and deleted the word “are” from the ground truth.

To calculate WER, we can use this formula: $(D+I+S)/N$. D is the number of deletions, I is the number of insertions, S is the number of substitutions and N is the number of words in the ground truth. In this example, the ASR output made 3 mistakes in total from 5 words in the ground truth. In this case, the WER would be $3 / 5 = 0.6$.

The LibriSpeech dataset

Next, we'll briefly touch on the LibriSpeech dataset. The LibriSpeech dataset is the most commonly used audio processing dataset in speech research. It was created by Vassil Panayotov and Daniel Povey in 2015 [3]. LibriSpeech consists of 960 hours of labelled speech data and is the standard benchmark for training and evaluating ASR systems.

The *dev-clean* dataset from LibriSpeech contains 5.4 hours of “clean” speech data. It’s generally used as a validation dataset. In the figure below, we show the transcription for one audio sample in the *dev-clean* dataset.

'HE DID NOT GO WITHIN A BLOCK OF THE HOUSE'

The transcription for one audio sample in the dev-clean dataset

What is wav2vec 2.0?

Now that we understand what an ASR system and the LibriSpeech dataset are, we’re ready to take a closer look at wav2vec 2.0.

What’s different about wav2vec 2.0?

ASR systems come in two flavors:

- The first are hybrid systems such as Kaldi [7] that train a deep acoustic model to predict phonemes from audio processed into Mel Frequency Cepstral Coefficients (MFCCs), combine the phonemes using a pronunciation dictionary and finally pick the most likely results using a language model (both count based LM and RNN based LM).
- The second are end-to-end systems using a deep neural network to predict words directly from the audio or MFCCs. Such systems like RNN-T [6] or wav2vec [1, 4] require a lot more training data and GPU resources for training.

Due to the massive data requirements of end-to-end systems only the biggest companies have used them to date. The data requirements also make it hard to train for new domains (even in the same language) and new languages or accents. Using a hybrid system, it is much easier to create a model for a new domain using minimal training data and a pronunciation dictionary with words added for that domain.

The promise of wav2vec 2.0 is pre-training without the supervised data using a large data set of recordings in the target domain. Afterwards, the model can be tuned using the supervised approach to maximize the accuracy. Wav2vec 2.0 shows that it’s possible to achieve low WER on LibriSpeech validation datasets using only ten minutes of

labelled audio data. Another option is to use the pre-trained model (such as the librispeech model) and just fine tune it for your domain with a few hours of labelled audio.

The architecture of wav2vec 2.0

The breakthrough wav2vec 2.0 achieved is in adopting the masked pre-training method of the massive language model BERT [8]. BERT masks a few words in each training sentence and the model trains by attempting to fill the gaps.

Instead of masking words, wav2vec 2.0 masks a part of the audio representation and requires the transformer network to fill in the gap.

The figure below shows the wav2vec 2.0 architecture with its two major components: CNN layers and transformer layers.

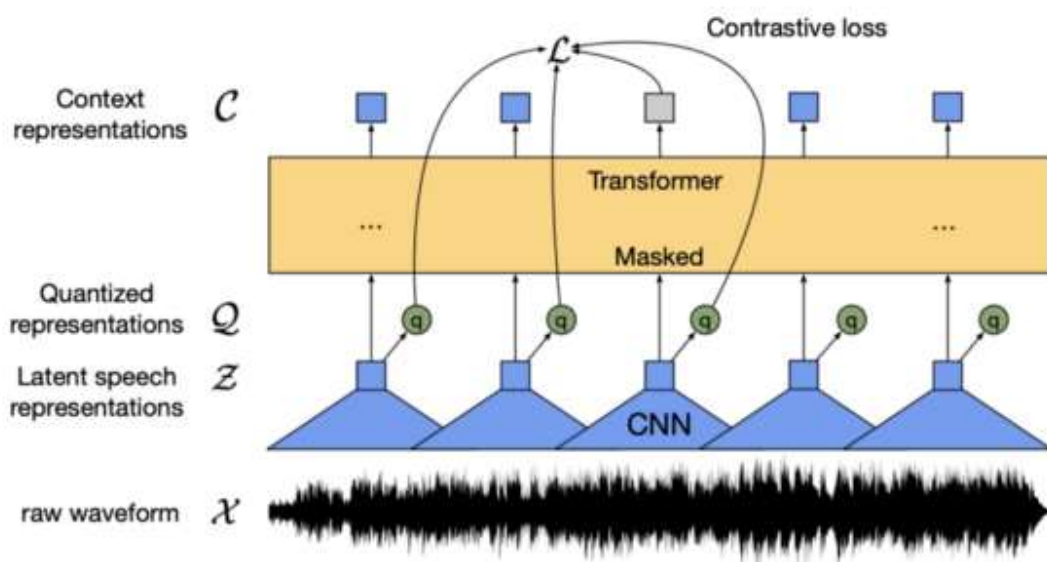


image credit: <https://arxiv.org/pdf/2006.11477.pdf>

Self-supervised learning

So how does self-supervised learning work in wav2vec 2.0? The raw audio waveform (X in the figure above) first passes through CNN layers, and we get latent speech representations (Z in the figure above). Now, two things happen in parallel:

1. We mask a random subset of Z , let's call it $\text{masked_}Z$. We pass $\text{masked_}Z$ into transformer layers. The output of the transformer layers is called context representations (C in the figure above).

2. We apply product quantization [5] on Z and get quantized representations (Q in the figure above).

We expect C to be close to Q over the masked parts. The “error” between C and Q over the masked parts is called the contrastive loss. Minimizing contrastive loss enables transformer layers to learn the structure inside latent speech representations (Z).

Where does wav2vec 2.0 fit in the big picture?

In the figure above, we saw that context representations were the output of transformer layers. Wav2vec 2.0 passes these context representations into a linear layer, followed by a softmax operation. The final output contains probability distributions over 32 tokens. A token can be a character, or it can represent word and sentence boundaries, as well as unknowns.

How do we convert these probability distributions into text? The answer is a decoder! The authors of wav2vec 2.0 used a beam search decoder. Below, we show you how to use a Viterbi decoder to convert the output of wav2vec 2.0 into text.

Similarity with word2vec

Word2vec [2] generates a feature vector for a given word, such that feature vectors of similar words have closer cosine similarity. Similar to word2vec, we can think of the wav2vec 2.0 output as a feature vector for an audio segment.

Using Python and PyTorch to build an end to end speech recognition system with wav2vec 2.0

Now, let’s look at how to create a working ASR with wav2vec 2.0 that generates text given audio waveforms from the LibriSpeech dataset. We used Python and PyTorch framework in our sample code snippets.

First, download the [wav2vec 2.0 model](#) and the [dev-clean dataset](#) from LibriSpeech. The *dev-clean* dataset contains 5.4 hours of “clean” speech data, and it’s generally used as a validation dataset.

```
1 model_path = "/home/models/wav2vec_big_960h.pt"
2 data_path = "/home/datasets/"
```

model_data_path.py hosted with ❤ by GitHub

[view raw](#)

In the code above, we declare `model_path`, which is the path to the wav2vec 2.0 model that we just downloaded. `data_path` is the path to the *dev-clean* dataset. Store it under `"/home/datasets/"`.

We mentioned in section 3.5 that wav2vec 2.0 outputs a probability distribution over 32 tokens. We convert these tokens to letters with the help from *ltr_dict.txt*. We download *ltr_dict.txt* from [here](#), and save it at `/home/ltr_dict.txt`.

You might notice that *ltr_dict.txt* contains only 28 letters and tokens. The remaining four tokens are `<s>`, `<pad>`, `</s>`, and `<unk>`, and they are added when we call `fairseq_mod.data.Dictionary.load()` with the path to *ltr_dict.txt*.

```
1 target_dict = fairseq_mod.data.Dictionary.load('ltr_dict.txt')
```

create_target_dict.py hosted with ❤ by GitHub

[view raw](#)

Now, create the wav2vec 2.0 model.

```
1 w2v = torch.load(model_path)
2 model = Wav2VecCtc.build_model(w2v["args"], target_dict)
3 model.load_state_dict(w2v["model"], strict=True)
```

create_wav2vec2.py hosted with ❤ by GitHub

[view raw](#)

In the code above, we first load from `model_path`. We get `w2v`, which contains the argument setup and the model's weights. Then, we build a wav2vecCTC object. wav2vecCTC is the model definition of wav2vec 2.0. Finally, we load weights into the model we just created.

We know that we need a decoder to convert the output of wav2vec 2.0 into text. Create a Viterbi decoder, as in code below.

```
decoder = W2lViterbiDecoder(target_dict)
```

Next, we need to create a data loader for our dataset. Luckily, torchaudio knows how to process the LibriSpeech dataset! To use it, we just need to call

[torchaudio.datasets.LIBRISPEECH](#)

```
1 dev_clean_librispeech_data = torchaudio.datasets.LIBRISPEECH(data_path, url='dev-clean', dow
2 data_loader = torch.utils.data.DataLoader(dev_clean_librispeech_data, batch_size=1, shuffle=
```

create_dev_clean_data_loader.py hosted with ❤ by GitHub

[view raw](#)

In the steps so far, we have created wav2vec 2.0, a Viterbi decoder, and the data loader. Now, we are ready to convert raw waveforms into text using wav2vec 2.0 and the decoder.

The code below shows how we pass one data sample into wav2vec 2.0. `encoder_input` is the data sample, a dictionary containing speech audio waveforms and other arguments that we need to pass into wav2vec 2.0. The `model` outputs `encoder_out`, representing logits over tokens at each time step. To get `encoder_out`, we project the output of wav2vec 2.0 into tokens through a linear layer. The dimension of `encoder_out` is $L \times B \times C$, where L is the sequence length, B is the batch size and C is the number of tokens.

As we saw in section 3.4, we know we need to pass probability distributions over tokens to the decoder to get transcribed texts. Since `encoder_out` are logits over tokens, we take the log softmax of these logits (through `model.get_normalized_probs`), and get `emissions`, which are probability distributions over tokens.

```
1 encoder_out = model(**encoder_input)
2 emissions = model.get_normalized_probs(encoder_out, log_probs=True)
3 emissions = emissions.transpose(0, 1).float().cpu().contiguous()
```

get_wav2vec2_output.py hosted with ❤ by GitHub

[view raw](#)

Next, we pass emissions into the decoder, like this:

```
1 decoder_out = decoder.decode(emissions)
```

get_wav2vec2_decoder_output.py hosted with ❤ by GitHub

[view raw](#)

In our third post in this series, we describe what happens inside the `decode` method. We need to do some post processing on `decoder_out` to finalize the output text, but we omit

those details here. Check out [post_process_sentence](#) if you are interested in knowing more.

That's it! We just finished processing one data sample. If you want to convert all data samples from the *dev-clean* dataset into texts and get a WER score, try [this notebook](#) and you should get a WER of 2.63%.

What's next?

In this post, we introduced the ASR system, as well as wav2vec 2.0. We also showed you how to get an ASR system working with wav2vec 2.0. Note that wav2vec 2.0 is a big model and its largest version has 317 million parameters! So, read our next post next to learn [how to compress wav2vec 2.0](#).

About Georgian R&D

Georgian is a fintech that invests in high-growth software companies.

At [Georgian](#), the R&D team works on building our platform that identifies and accelerates the best growth stage software companies. As part of this work, we take the latest [AI research](#) and use it to help solve the business challenges of the companies where we are investors. We then create reusable toolkits so that it's easier for our other companies to adopt these techniques.

We wrote this series of posts after an engagement where we collaborated closely with the team at [Chorus](#). Chorus is a conversation intelligence platform that uses AI to analyze sales calls to drive team performance.

Take a look at our open opportunities if you're interested in a [career at Georgian](#).

References

- [1] Baevski et al. (2020). Wav2vec 2.0: A Framework for Self-Supervised Learning of Speech Representations. <https://arxiv.org/abs/2006.11477>
- [2] Mikolov et al. (2013). Efficient Estimation of Word Representations in Vector Space. <https://arxiv.org/abs/1301.3781>
- [3] Panayotov et al. (2015). Librispeech: an asr corpus based on public domain audio books. <https://ieeexplore.ieee.org/document/7178964>

[4] Schneider et al. (2019). wav2vec: Unsupervised Pre-training for Speech Recognition. <https://arxiv.org/abs/1904.05862>

[5] Jegou et al. (2011). Product quantization for nearest neighbor search. IEEE Trans. Pattern Anal. Mach. Intell., 33(1):117–128

[6] Alex Graves (2012) Sequence Transduction with Recurrent Neural Networks. <https://arxiv.org/pdf/1211.3711.pdf>

[7] Povey et al. (2011) The Kaldi Speech Recognition Toolkit. IEEE 2011 Workshop on Automatic Speech Recognition and Understanding. <https://kaldi-asr.org/doc/about.html>

[8] Devlin et al. (2018) BERT: Pre-training of Deep Bidirectional Transformers for Language Understanding. <https://arxiv.org/abs/1810.04805>

Sign up for Georgian Impact Blog Updates

By Georgian Impact Blog

Learn about the latest news and updates from Georgian's R&D team. [Take a look.](#)

Your email

Get this newsletter

By signing up, you will create a Medium account if you don't already have one. Review our [Privacy Policy](#) for more information about our privacy practices.

Asr System Speech Recognition NLP Machine Learning Python

About Write Help Legal

Get the Medium app

